

**Course: Fundamentals of Generative AI and Large Language
Models: Theory and Practice**

**Week 3: Variational Autoencoders (VAEs):
Foundations**

Today's Session Overview

Variational Autoencoder:

- Architecture and working of the Encoder

NPTEL

Motivation for Variational Autoencoders: **Generative Modelling**

Autoencoder learned **reconstruction**.

VAE is introduced because we want generation.

Variational Autoencoders (VAEs)

Given a dataset of N samples:

$$X = \{x^{(1)}, x^{(2)}, \dots, x^{(N)}\}$$

Each class follow its own class-conditional data distribution

Samples belonging to the same class follow a similar pattern.

For example, Iris-Setosa samples have one type of feature pattern, while Iris-Virginica samples have another type of feature pattern.

Since this is an unsupervised learning model, class labels are not used during training. Therefore, the model must understand the underlying pattern of the data in order to generate new data samples.

But:

- We do not know the true data distribution $p_{data}(x)$.
- We do not know its exact functional form, such as Gaussian, Bernoulli, Binomial, Poisson, etc., or its parameters.
- We only have a finite set of samples, that is, the dataset drawn from this unknown distribution.

sepal length	sepal width	petal length	petal width	class
5.1	3.5	1.4	0.2	Iris-setosa
4.9	3.0	1.4	0.2	Iris-setosa
4.7	3.2	1.3	0.2	Iris-setosa
4.6	3.1	1.5	0.2	Iris-setosa
5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...
6.7	3.0	5.2	2.3	Iris-virginica
6.3	2.5	5.0	1.9	Iris-virginica
6.5	3.0	5.2	2.0	Iris-virginica
6.2	3.4	5.4	2.3	Iris-virginica
5.9	3.0	5.1	1.8	Iris-virginica

Variational Autoencoders (VAEs)

Goal (Generative Modelling)

The goal of a generative model is to learn the underlying data distribution and generate new samples x_{new} that look similar to the training data.

The original data distribution $p_{data}(x)$ is unknown. The model only observes a finite set of training samples drawn from this distribution.

Therefore, the model tries to learn an approximate distribution $p_{\theta}(x)$ that is close to the true data distribution.

$$p_{\theta}(x) \approx p_{data}(x)$$

Here, $p_{\theta}(x)$ means the distribution learned by the neural network using parameters θ , such as weights and biases.

At the beginning of training, the learned distribution $p_{\theta}(x)$ may be very different from the true data distribution $p_{data}(x)$.

As training progresses, the model parameters θ are updated, and $p_{\theta}(x)$ gradually becomes closer to $p_{data}(x)$.

After learning this distribution, the model can generate new samples as:

$$x_{new} \sim p_{\theta}(x)$$

Variational Autoencoders (VAEs)

Two distributions

1. Data space distribution:

$$p_{data}(x)$$

This represents the unknown distribution of the given data samples.

$$x_1, x_2, x_3, \dots, x_4$$
$$p_{data}(x)$$

2. Latent space distribution:

$$p(z)$$

x and z live in different spaces means they have different dimensionality and structure. The input (x) belongs to the data space, whereas (z) belongs to the latent space. Therefore, we need probabilistic mappings to connect the data space and the latent space.

Two key probabilistic mappings in VAE

1. Approximate Posterior distribution (Encoder)

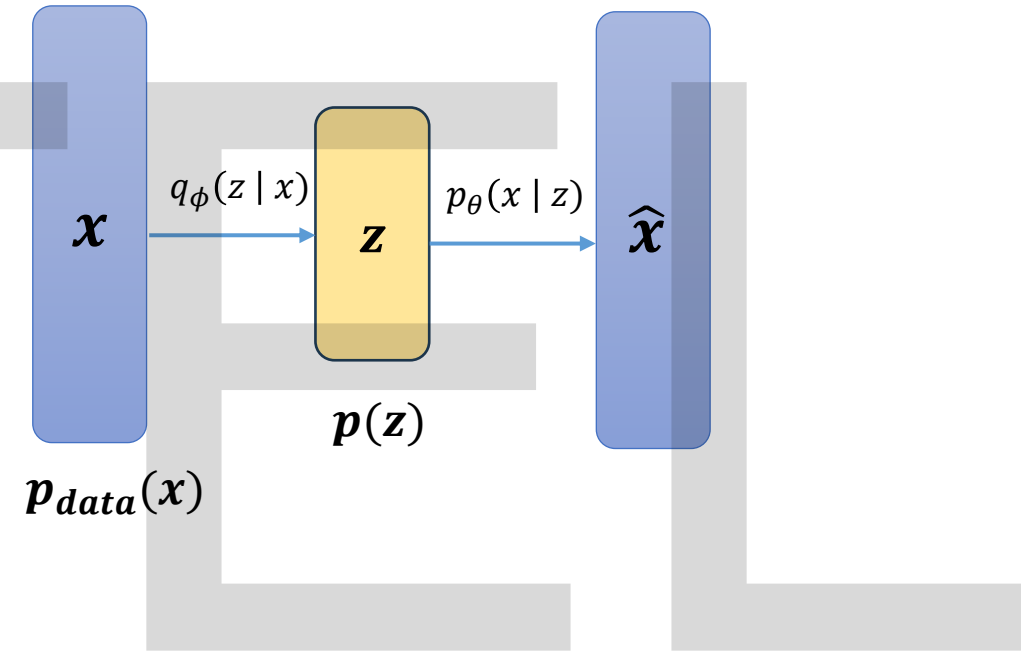
The encoder does **not directly learn** the true posterior as it is difficult to compute:

$$p(z | x)$$

Instead, it learns an **approximate posterior**:

$$q_{\phi}(z | x)$$

Encoder maps an input data point x to a distribution over latent variable z
The probability of the latent variable z given the data sample x



2. Likelihood distribution (Decoder)

$$p_{\theta}(x | z)$$

Decoder maps latent variable z back to data space

The probability of generating data x given the latent variable z

Variational Autoencoders (VAEs): Encoder

The primary objective of the encoder is to infer the posterior distribution of the latent variable z given the input x :

The posterior $p(z | x)$ depends on:
$$p(z | x) = \frac{p(x | z) \cdot p(z)}{p(x)}, \text{ from Bayes rule}$$

$p(z | x)$ = true posterior distribution

$p(x | z)$ = likelihood distribution

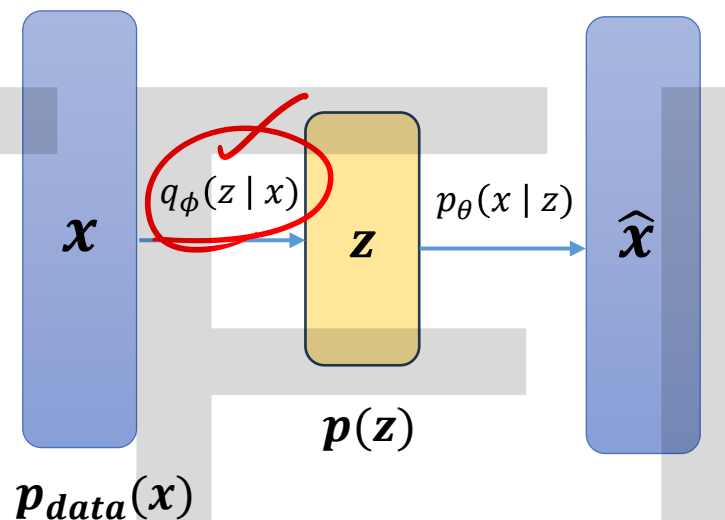
$p(z)$ = prior distribution

$p(x)$ = evidence

$$p(x) = \int p(x | z) \cdot p(z) dz$$

Computing $p(x)$ is very hard. But why?

Because it requires integration over all possible values of the latent variable z .



Therefore, in VAE, instead of directly computing the true posterior $p(z | x)$, it approximate it using another distribution:

$q_{\phi}(z | x)$
such that: $q_{\phi}(z | x) \approx p(z | x)$

The encoder network learns this approximate posterior distribution.

Variational Autoencoders (VAEs): Encoder

In a VAE, the latent variable is not a single number, but a vector:

$$\mathbf{z} = [z_1, z_2, \dots, z_d]$$

Each z_i is called a latent dimension (or latent variable).

Suppose if the input data has 100 features, then

$$100 \rightarrow \text{Encoder} \rightarrow 25.$$

$\therefore$ The latent variable $\mathbf{z}$ is not a single number, but a 25-dimensional vector. $\mathbf{z} = [z_1, z_2, \dots, z_{25}]$

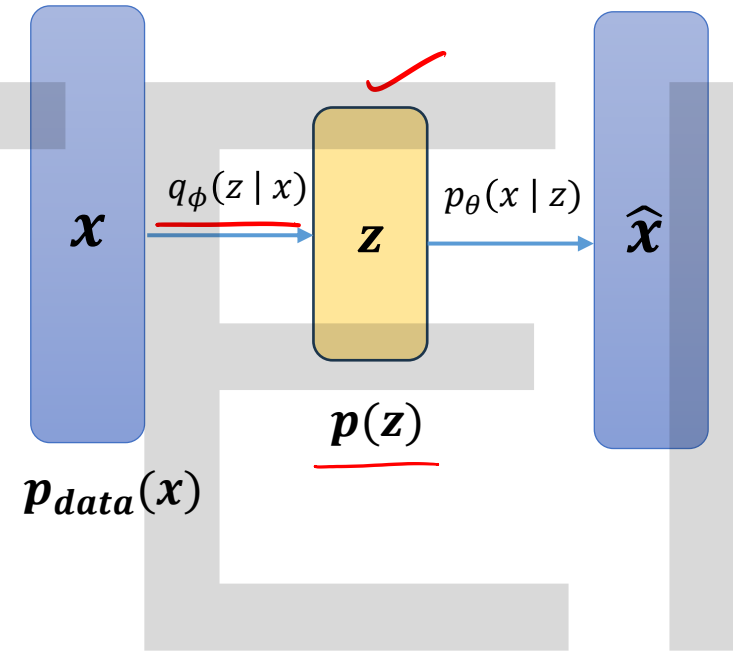
The purpose of the encoder is to learn a meaningful latent representation of the input data in this lower-dimensional latent space. However, we do not know the true structure of the latent space.

$\therefore$ In VAE, the prior distribution $p(\mathbf{z})$ is chosen to be a standard normal distribution (Gaussian).

$$p(\mathbf{z}_i) = \mathcal{N}(\mathbf{0}, \mathbf{1}) \rightarrow \text{Prior probability}$$

This enforces a well-structured and continuous latent space.

- ✓ Gaussian is simple and mathematically convenient.
- ✓ Sampling becomes easy



Variational Autoencoders (VAEs): Encoder

In VAE, the encoder goal is to learn a distribution over the latent variable:

$$q_{\phi}(z | x)$$

Instead of allowing this distribution to be any complex distribution, we assume it to be Gaussian:

$$q_{\phi}(z | x) = \mathcal{N}(\mu_{\phi}(x), \sigma_{\phi}^2(x))$$

The prior $p(z)$ serves as a reference distribution for $q_{\phi}(z | x)$. $\therefore$ from input only we get the latent space.

So choosing $q_{\phi}(z | x)$ also as Gaussian makes it easier to compare $q_{\phi}(z | x)$ with $p(z)$

A Gaussian distribution can be represented using only two parameters:

μ and σ^2

So the encoder only needs to output:

$$\mu_{\phi}(x), \sigma_{\phi}^2(x)$$

That makes the model easier to train.

Once the encoder gives $\mu_{\phi}(x)$ and $\sigma_{\phi}^2(x)$, we can sample z from that Gaussian distribution.

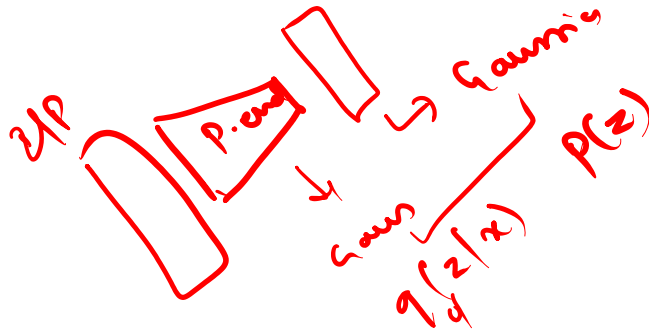
$$z \sim \mathcal{N}(\mu_{\phi}(x), \sigma_{\phi}^2(x))$$

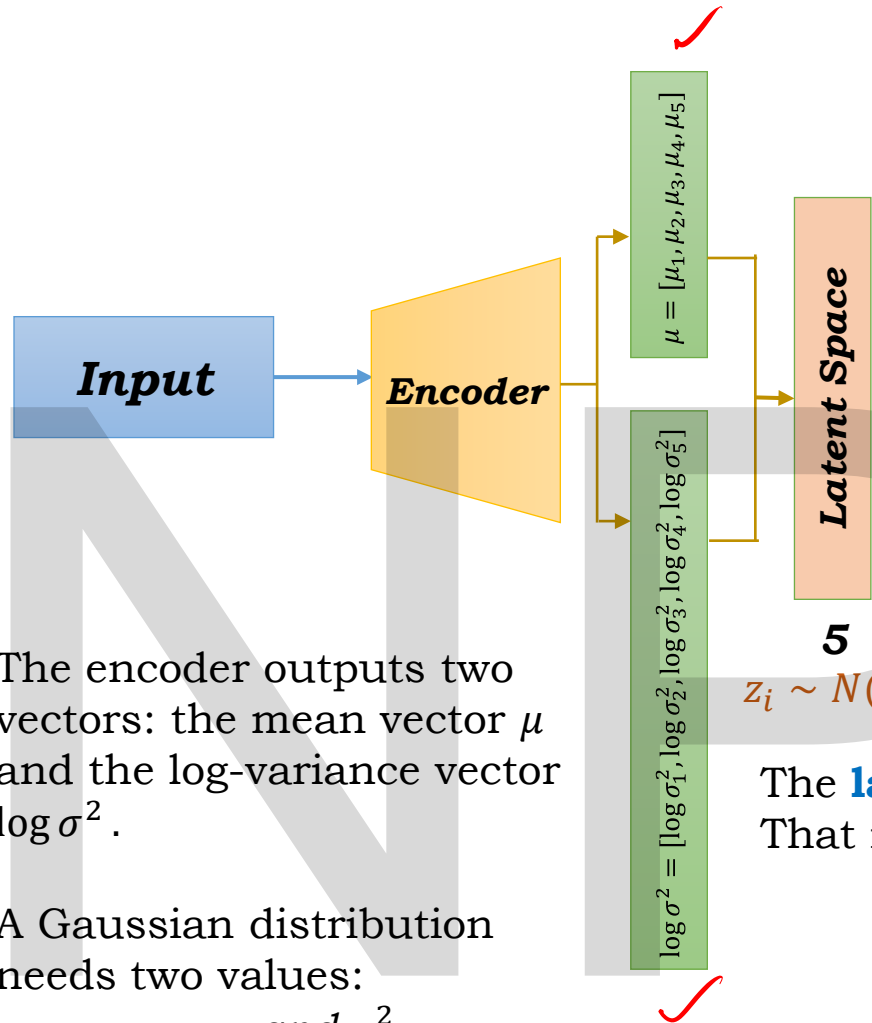
$q_{\phi}(z | x)$ and $p(z)$ are gaussian

Who ensures both distributions are Gaussian?

$$D_{KL}(q_{\phi}(z | x) \parallel p(z))$$

KL divergence term ensures that the learned encoder distribution remains close to this prior.





The encoder outputs two vectors: the mean vector μ and the log-variance vector $\log \sigma^2$.

A Gaussian distribution needs two values:
 μ_i and σ_i^2

So, convert log-variance to variance for the corresponding latent dimension.

$$\sigma_i^2 = \exp(\log \sigma_i^2)$$

$$\begin{aligned} (\mu_1, \sigma_1^2) &\Rightarrow z_1 \sim N(\mu_1, \sigma_1^2) \\ (\mu_2, \sigma_2^2) &\Rightarrow z_2 \sim N(\mu_2, \sigma_2^2) \\ (\mu_3, \sigma_3^2) &\Rightarrow z_3 \sim N(\mu_3, \sigma_3^2) \\ (\mu_4, \sigma_4^2) &\Rightarrow z_4 \sim N(\mu_4, \sigma_4^2) \\ (\mu_5, \sigma_5^2) &\Rightarrow z_5 \sim N(\mu_5, \sigma_5^2) \end{aligned}$$

The **latent space dimension is chosen as 5**.

That means the latent vector is:

$$z = [z_1, z_2, z_3, z_4, z_5]$$

So for 5 latent dimensions, the encoder outputs:

$$\mu = [\mu_1, \mu_2, \mu_3, \mu_4, \mu_5]$$

and

$$\log \sigma^2 = [\log \sigma_1^2, \log \sigma_2^2, \log \sigma_3^2, \log \sigma_4^2, \log \sigma_5^2]$$

Therefore, the sampled latent vector is:

$$z = [0.65, -0.10, 0.80, 0.42, -0.70]$$

Mean vector:

$$\mu = [0.5, -0.2, 1.0, 0.3, -0.5]$$

Log-variance vector:

$$\log \sigma^2 = [-1.386, -2.303, -0.693, -1.833, -1.386]$$

Convert log-variance into variance:

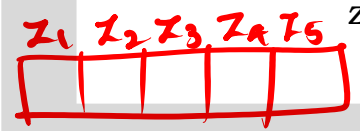
$$\sigma^2 = \exp(\log \sigma^2)$$

Therefore:

$$\sigma^2 = [0.25, 0.10, 0.50, 0.16, 0.25]$$

Thus, the five latent variables follow:

$$\begin{aligned} z_1 &\sim N(0.5, 0.25) \\ z_2 &\sim N(-0.2, 0.10) \\ z_3 &\sim N(1.0, 0.50) \\ z_4 &\sim N(0.3, 0.16) \\ z_5 &\sim N(-0.5, 0.25) \end{aligned}$$



Now sample one value from each Gaussian distribution.

Suppose the sampled values are:

$$\begin{aligned} z_1 &= 0.65 \\ z_2 &= -0.10 \\ z_3 &= 0.80 \\ z_4 &= 0.42 \\ z_5 &= -0.70 \end{aligned}$$

Next Session

- Decoder part of VAE
- Loss function of VAE

NIPTELL

Thank you